

TenderWatch

Агрегатор тендеров и закупок с системой подписок и алертов

Техническое задание

Стек: PHP 8.3+, Yii3, PostgreSQL, RabbitMQ, Redis, Docker

Версия документа: 1.0

Дата: 18 июня 2026

Содержание

1. Цель и назначение проекта
2. Архитектура
3. Технический стек
4. Доменная модель
5. Прицельная демонстрация механик Yii3
6. Схема базы данных (основные таблицы)
7. REST API (основные эндпоинты)
8. План по неделям (3–4 недели)
9. Критерии готовности (Definition of Done)
10. Риски и опции упрощения

1. Цель и назначение проекта

TenderWatch — пет-проект для портфолио на Yii3, агрегирующий тендеры/закупки из нескольких источников и оповещающий пользователей о новых позициях, подходящих под их подписки (категория, бюджет, регион, ключевые слова).

Проект решает две задачи одновременно:

1. Демонстрирует архитектурную зрелость: DDD-разделение на bounded context'ы, чистые границы между доменом и инфраструктурой, явный DI.
2. Закрывает типичные провалы на собеседованиях по Yii3 — практическое применение DI-контейнера, middleware pipeline, событий и системы конфигурации пакетов, а не только теоретическое знание.

1.1 Целевая аудитория демо

- Технические интервьюеры — видят архитектуру, тесты, осознанные решения.
- HR / нетехнические собеседники — видят рабочий продукт с понятной пользой (находить тендеры/заказы выгоднее конкурентов).
- Сам автор — годная база для последующего расширения (например, в реальный сервис для фрилансеров).

1.2 Ограничения объёма (3–4 недели, один разработчик)

- Один реальный источник данных + один синтетический генератор для нагрузочного тестирования.
- Без полноценной биллинговой системы — подписки бесплатные, тарифы можно оставить заглушкой.
- Фронтенд — минимальный (Vue 3 SPA или HTMX), фокус на бэкенде.

2. Архитектура

Проект построен по слоистой архитектуре (Clean/Hexagonal) с явным разделением на bounded context'ы. Домен не знает о Yii3, Doctrine/Cycle ORM или транспорте — только PHP-интерфейсы.

2.1 Bounded contexts

Контекст	Зона ответственности	Ключевые объекты
Tenders	Хранение и нормализация тендеров	Tender (arperat), Category, Deadline, Money (VO)
Subscriptions	Правила фильтрации пользователей	Subscription, FilterCriteria (VO), MatchRule
Matching	Сопоставление тендеров с подписками	MatchingEngine, MatchResult
Notifications	Доставка уведомлений	NotificationChannel, Template,

Контекст	Зона ответственности	Ключевые объекты
		DeliveryLog
Ingestion	Получение данных из внешних источников	SourceConnector, RawTenderDTO, Normalizer
Identity	Пользователи и аутентификация	User, ApiToken

2.2 Слои внутри каждого контекста

```
src/Tenders/
├── Domain/           → Entity, ValueObject, Repository-интерфейсы, Domain-события
├── Application/      → UseCase / Command+Handler, DTO
├── Infrastructure/   → реализация репозитория (Cycle/DBAL), внешние клиенты
└── Presentation/    → PSR-15 контроллеры, маппинг запрос/ответ
```

Domain-слой не зависит от Yii3 — только от PHP и собственных интерфейсов. Это позволяет на интервью явно показать, где проходит граница и почему её важно держать (тестируемость, независимость от фреймворка).

2.3 Поток обработки тендера (упрощённо)

1. Ingestion: воркер забирает данные из источника (HTTP API/файл) → DTO.
2. Normalizer превращает DTO во входной объект домена → Tender создаётся/обновляется через репозиторий.
3. При сохранении нового тендера диспетчер событий публикует TenderCreated.
4. Listener в контексте Matching запускает MatchingEngine против активных подписок.
5. При совпадении публикуется SubscriptionMatched → ставится job в очередь на отправку уведомления.
6. Notification-воркер забирает job, выбирает канал (email/Telegram/webhook) и отправляет.

3. Технический стек

Слой	Технология	Назначение
Язык/фреймворк	PHP 8.3+, Yii3 (yiisoft/*)	Базовая платформа
DI	yiisoft/di	Контейнер зависимостей, конфигурация биндингов
HTTP	yiisoft/http, PSR-15 middleware	Роутинг, middleware pipeline
ORM/DBAL	Cycle ORM или yiisoft/db	Доступ к PostgreSQL
БД	PostgreSQL 16	Основное хранилище
Кэш	yiisoft/cache + Redis	Кэш matching-результатов, rate limiting
Очереди	yiisoft/queue + RabbitMQ	Асинхронный ingestion и доставка уведомлений
События	yiisoft/event-dispatcher	Domain events → listeners
Почта	Symfony Mailer	Email-канал уведомлений
Telegram	HTTP-клиент (yiisoft/http-client)	Telegram Bot API канал
Тесты	PHPUnit, Codeception (опц.)	Unit + функциональные тесты
Контейнеризация	Docker, docker-compose	Локальный запуск и деплой
CI	GitHub Actions	Линт, статанализ (PHPStan), тесты

4. Доменная модель

4.1 Контекст Tenders

- Tender (арперат): id, title, description, category, budget (Money VO), region, deadline (DateRange VO), sourceId, status, createdAt
- Category: id, name, slug, parentId (древовидная структура)
- Money (VO): amount, currency — immutable, со встроенной валидацией
- DateRange (VO): publishedAt, deadlineAt — с методом isExpired()

4.2 Контекст Subscriptions

- Subscription: id, userId, name, criteria (FilterCriteria VO), channels[], isActive
- FilterCriteria (VO): categories[], minBudget, maxBudget, regions[], keywords[] — immutable, с фабричным методом fromArray()

4.3 Контекст Matching

- MatchingEngine: метод match(Tender, Subscription[]): MatchResult[]
- Реализация через паттерн Specification — каждый критерий (категория, бюджет, регион, ключевые слова) — отдельная Specification с composable and()/or()

4.4 Domain-события

Событие	Когда публикуется	Слушатели
TenderCreated	Новый тендер сохранён в Tenders	MatchingEngine listener
TenderUpdated	Изменились условия тендера (например, бюджет)	Повторный matching, инвалидация кэша
SubscriptionMatched	MatchingEngine нашёл совпадение	Постановка job в очередь Notifications
NotificationSent	Канал успешно доставил сообщение	DeliveryLog, метрики
NotificationFailed	Сбой доставки (например, Telegram недоступен)	Retry-политика, алерт в лог

5. Прицельная демонстрация механик Yii3

Раздел существует специально, чтобы закрыть конкретные слабые места, которые обычно проявляются на собеседованиях после сильного практического опыта на Laravel/Symfony.

5.1 DI-контейнер и конфигурация

- Явные биндинги интерфейсов репозиториев и сервисов в `config/common/di/tenders.php`, `di/subscriptions.php` и т.д. — отдельный файл на контекст, не один общий.
- Использование `config groups` (`params`, `di`, `events`, `routes`) и демонстрация, как Yii3 мерджит конфиги из разных пакетов/контекстов в один runtime-конфиг.
- Autowiring там, где он очевиден, и явные фабрики (Closure-биндинги) там, где нужна сложная инициализация (например, HTTP-клиент Telegram с базовым URL и токеном из ENV).

5.2 PSR-15 Middleware pipeline

- `ApiKeyAuthMiddleware` — аутентификация внешних потребителей API по заголовку.
- `RateLimitMiddleware` — лимит запросов на основе Redis-счётчика, демонстрация Decorator/Chain of Responsibility в явном виде.
- `ErrorHandlingMiddleware` — единый формат ошибок (`problem+json`).
- Порядок объявления pipeline явно прописан в `config/web/routes.php` — повод объяснить на интервью, как и почему порядок middleware влияет на поведение.

5.3 Event Dispatcher

- Чёткая граница: домен публикует событие через интерфейс `EventDispatcherInterface` (PSR-14), не зная о Yii3 listener provider.
- Listener provider конфигурируется в `config/common/events.php` — маппинг событие → listener[].

5.4 Что заранее проговорить на интервью

- Чем Yii3 принципиально отличается от Yii2 (отказ от Active Record как обязательного паттерна, PSR-стандарты, неймспейсы пакетов `yii\*`, отсутствие глобального `Yii::$app`).
- Почему конфигурация в Yii3 — "explicit over magic" в сравнении с Laravel service container auto-resolution.
- Как тестировать код, завязанный на DI-контейнер (PSR-11 ContainerInterface в тестах через тестовые конфиги).

6. Схема базы данных (основные таблицы)

Таблица	Ключевые поля	Связи
tenders	id, title, category_id, budget_amount, budget_currency, region, deadline_at, source_id, status	FK → categories, sources
categories	id, name, slug, parent_id	self-FK (дерево)
sources	id, name, type, config_json, last_sync_at	—
subscriptions	id, user_id, name, criteria_json, is_active	FK → users
subscription_channels	id, subscription_id, channel_type, target	FK → subscriptions
matches	id, tender_id, subscription_id, matched_at	FK → tenders, subscriptions
notifications_log	id, match_id, channel_type, status, sent_at, error	FK → matches
users	id, email, password_hash, telegram_chat_id	—
api_tokens	id, user_id, token_hash, expires_at	FK → users

7. REST API (основные эндпоинты)

Метод и путь	Описание	Auth
GET /api/v1/tenders	Список тендеров с фильтрами (category, region, budget, q)	API key
GET /api/v1/tenders/{id}	Детали тендера	API key
POST /api/v1/subscriptions	Создать подписку	Bearer (user)
GET /api/v1/subscriptions	Список подписок пользователя	Bearer (user)
PATCH /api/v1/subscriptions/{id}	Изменить критерии/каналы подписки	Bearer (user)
DELETE /api/v1/subscriptions/{id}	Деактивировать подписку	Bearer (user)
POST /api/v1/webhooks/telegram	Webhook от Telegram Bot API	Telegram secret token
GET /api/v1/health	Health-check (БД, очередь, кэш)	—

Формат ответа — единый конверт: успех { success: true, data, meta }, ошибка { success: false, message, errors }. Документация — OpenAPI 3.0 спека (генерация через атрибуты или статический yaml-файл).

8. План по неделям (3–4 недели)

Неделя 1 — Фундамент и контекст Tenders

- Инициализация Yii3-приложения, структура каталогов по контекстам.
- Docker-compose (PHP, PostgreSQL, Redis, RabbitMQ).
- Domain-модель Tender, Category, Money/DateRange VO + unit-тесты домена.
- Миграции БД, репозитории (Infrastructure), DI-конфигурация контекста Tenders.
- Базовый CRUD API для тендеров и категорий (без auth).

Неделя 2 — Subscriptions, Matching, API hardening

- Domain-модель Subscription, FilterCriteria VO.
- MatchingEngine на паттерне Specification + unit-тесты на разные комбинации критериев.
- REST API для подписок, PSR-15 middleware: ApiKeyAuth, RateLimit, ErrorHandling.
- OpenAPI-спека, базовые функциональные тесты API.

Неделя 3 — Events, очереди, нотификации, ingestion

- Event Dispatcher: TenderCreated → MatchingEngine listener → SubscriptionMatched.
- yiiisoft/queue + RabbitMQ: воркер постановки/обработки уведомлений.
- Email-канал (Symfony Mailer) и Telegram-канал (бот, webhook, отправка сообщений).
- Ingestion-воркер с одним реальным источником (например, открытые API госзакупок) + Normalizer.
- Cron/scheduler для периодического запуска ingestion.

Неделя 4 — Полировка, демо, деплой

- Минимальный фронтенд/дашборд (список тендеров, форма подписки, лог уведомлений).
- Нагрузочный тест MatchingEngine на синтетических данных (сколько подписок/тендеров в секунду).
- README с архитектурной диаграммой, инструкцией запуска, описанием решений (ADR-style).
- CI: GitHub Actions (PHPStan, PHPUnit, lint).
- Деплой демо-версии (VPS/Render/Railway) для показа на собеседовании.

9. Критерии готовности (Definition of Done)

- Домен покрыт unit-тестами (агрегаты, VO, MatchingEngine) — без обращения к БД.
- Минимум 2 канала уведомлений работают end-to-end (email + Telegram).
- API задокументирован (OpenAPI) и покрыт функциональными тестами на основные сценарии.

- README объясняет архитектурные решения и явно перечисляет, какие механики Yii3 где применены.
- Проект поднимается одной командой docker-compose up и доступен для демо.
- PHPStan уровень не ниже 6 без подавленных ошибок.

10. Риски и опции упрощения

Риск	Митигация
Не хватит времени на все 4 недели	Урезать до email-канала без Telegram, отложить ingestion-источник до синтетического генератора
Yii3-пакеты нестабильны/мало документации	Зафиксировать конкретные версии в composer.json в начале, не обновлять посреди разработки
MatchingEngine станет узким местом при росте данных	Заложить индексы в БД заранее, кэш через Redis на повторяющиеся подписки
Сложность Telegram webhook в локальной разработке	Использовать ngrok/Cloudflare Tunnel для локального тестирования вебхука

*Документ подготовлен как рабочее ТЗ для самостоятельной разработки в рамках портфолио.
Структура контекстов и сроки — отправная точка, может корректироваться по ходу разработки.*